

LAPORAN PRAKTIKUM
Komunikasi Asinkron dan WebSockets

LAB SESSION 04



Disusun Oleh:

NAMA : Fandika Nugraha

NPM : 25782097

KELAS : TRI – 3D

MATA KULIAH : Teknologi IoT II

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

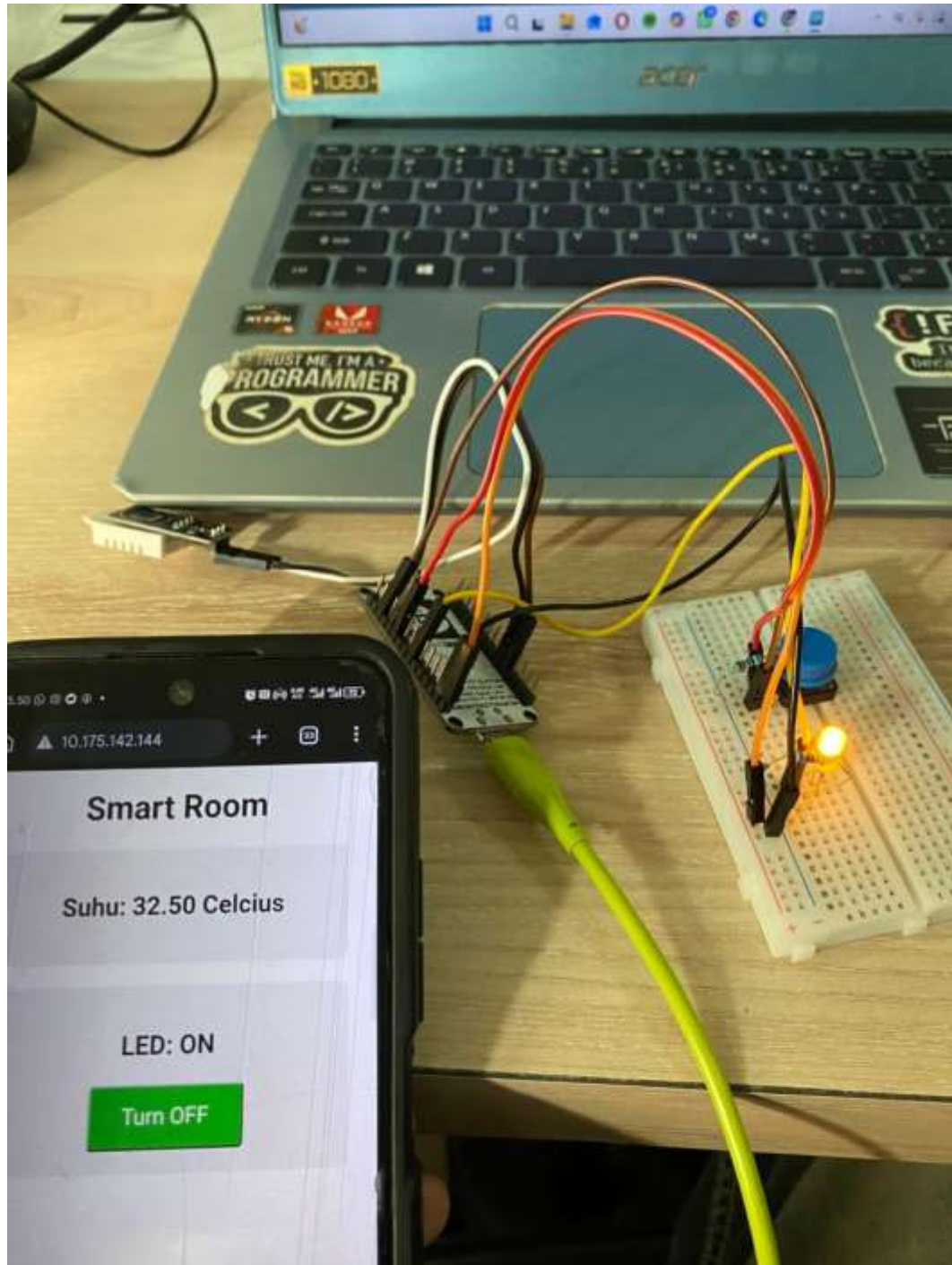
Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2025/2026

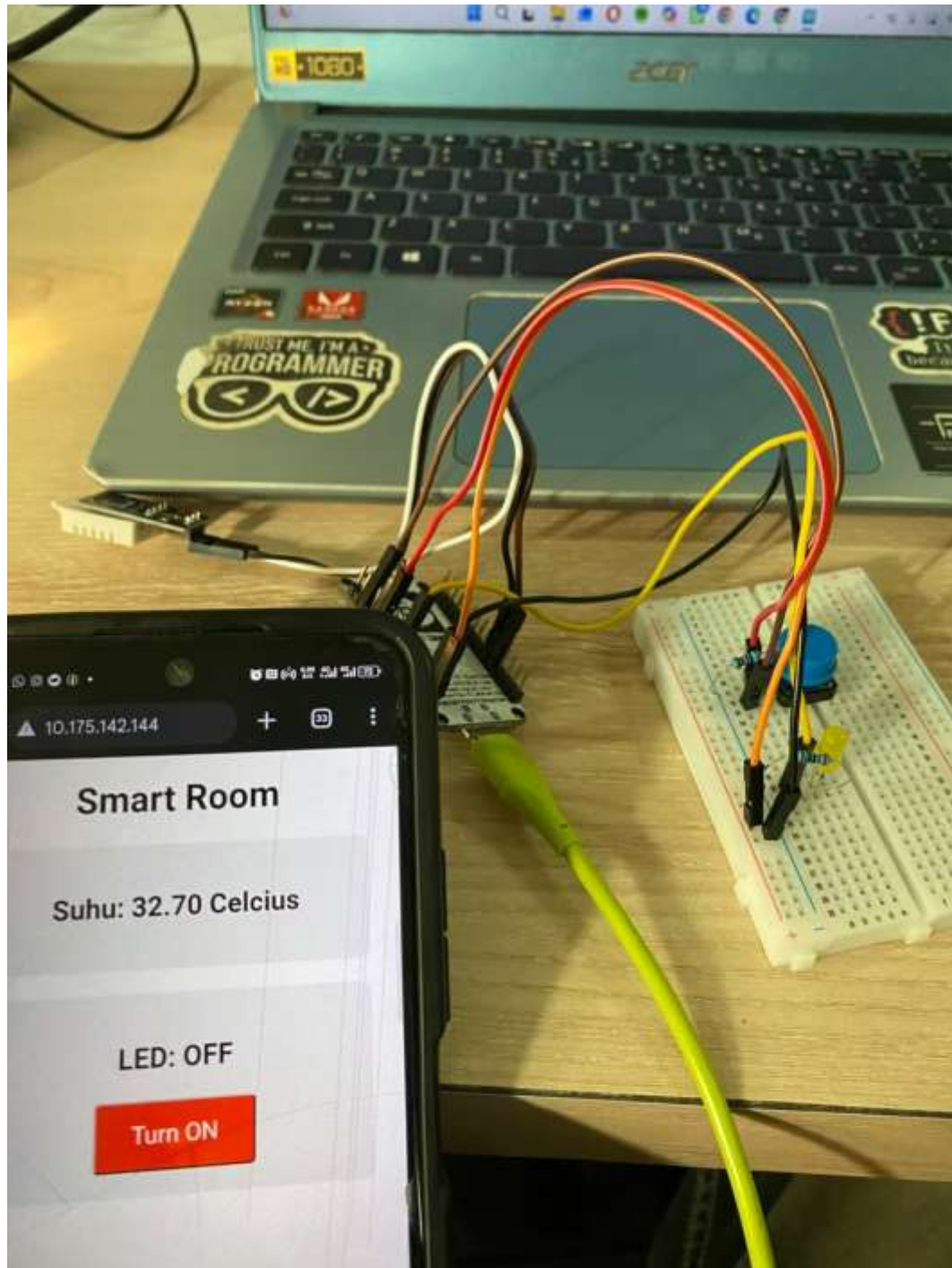
4.5 Praktikum

4.5.1 Praktikum 1: *Real-Time Dashboard* & Sinkronisasi Tombol Fisik

Tampilan kondisi LED ON:



Tampilan kondisi LED OFF:



Tampilan Serial Monitor:

```
1 #include <ESP8266WiFi.h>
2 #include <ESPAsyncTCP.h>
3 #include <ESPAsyncWebServer.h>
4 #include <DHT.h>
5
6 const char* ssid = "TECNO POMA 5";
7 const char* password = "Fandik11";
8
9 // Konfigurasi Pin
10 const byte dhtPin = 2; // D4 (GPIO 2)
```

Output Serial Monitor x

Message (Enter to send message to NodeMCU v0 (ESP-12E Module) on COM5)

New Line 115200 baud

```
*****
Koneksi Wi-Fi Berhasil!
Alamat IP Web Server: 10.175.142.144
HTTP server telah aktif
*****
Koneksi Wi-Fi Berhasil!
Alamat IP Web Server: 10.175.142.144
HTTP server telah aktif
*****
IP Address: 10.175.142.144
Client WebSocket #1 terhubung
Client WebSocket #1 terputus
Client WebSocket #2 terhubung
Client WebSocket #2 terputus
```

4.6 Latihan

Latihan 1 (Teori Jaringan): Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh* *HTTP* *HTML* yang Anda gunakan pada Tugas Praktikum Modul 3?

Jawaban:

WebSockets itu kayak panggilan telepon: Sekali nyambung (*handshake*), jalurnya dibiarkan terbuka terus. Kalau ada data baru (misal suhu berubah), datanya tinggal dilempar lewat jalur itu tanpa perlu kirim kepala surat (*header HTTP*) yang besar berulang-ulang. Makanya kuota/bandwidth hemat dan layar HP nggak pakai acara *reload* atau kedip-kedip.

Meta Refresh itu kayak tamu yang bolak-balik keluar-masuk rumah: Setiap beberapa detik, browser dipaksa megang gagang pintu, buka pintu, dan mengunduh seluruh isi halaman web dari nol lagi. Jelas bikin boros data dan layarnya kedip setiap kali halaman disegarkan.

Latihan 2 (Analisis Waktu/Timer): Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan *NodeMCU*) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

Jawaban:

Kalau Anda nekat naruh sintaks `delay(3000)` di dalam `loop()`, mikrokontrolernya bakal "mati suri" alias macet total selama 3 detik.

- Akibatnya? Tombol di web yang Anda pencet bakal diacuhkan, koneksi WebSocket bisa putus karena *timeout*, atau malah ESP8266-nya *restart* sendiri karena mengira programnya *hang* (tersangkut). Makanya kita wajib pakai pewaktuan non-blocking kayak `millis()`.

Latihan 3 (Modifikasi Front-End): Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML (`<span id="...">`) apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

Jawaban:

Di bagian *front-end*, elemen-elemen HTML yang diincar sama JavaScript buat diubah-ubah isinya secara *real-time* itu pakai ID ini:

- **id="temp"**: Buat nampilin angka suhu.
- **id="hum" atau id="humValue"**: Buat nampilin angka kelembapan.
- **id="ledStatus"**: Buat nulis status LED apakah lagi ON atau OFF.
- **id="btn"**: Buat ngubah tulisan di tombol (misal dari "Nyalakan LED" jadi "Matikan LED") sekaligus ganti warna tombolnya.

Latihan 4 (Pengembangan Telemetry Sensor - Kelembapan):

Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:

Sisi Back-End (C++): Buat variabel penampung (misal: `String currentHum = "--"`). Baca nilai kelembapan menggunakan fungsi `dht.readHumidity()` di dalam siklus timer *non-blocking* `millis()`. Perbarui fungsi `notifyClients()` agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: `{"led": "...", "suhu": "...", "hum": "..."}).`

Sisi Front-End (HTML & JavaScript): Tambahkan elemen kartu (`<div class="card">`) baru pada kerangka HTML sebagai penampil kelembapan (misal:

<h2>Kelembapan: -- %</h2>). Kemudian, modifikasi fungsi JavaScript onMessage(event) agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen humValue. Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

Jawaban:

1. Di Sisi Back-End (C++)

Bikin variabel penampungnya, lalu baca sensornya pakai fungsi dht.readHumidity() di dalam timer millis():

```
1 String currentHum = "--"; // sudah penampung data kelembapan
2
3 // Konversi data "hum" ke dalam format JSON
4 void notifyClients() {
5   String jsonString = "{\"temp\":\"" + currentTemp + "\", \"hum\":\"" + currentHum + "\", \"led\":\"" + (ledState ? "ON" : "OFF") + "\"}";
6   ws.textAll(jsonString);
7 }
8
9 // Di dalam void loop() (timer non-blocking)
10 if (millis() - lastSensorTime > 3000) {
11   lastSensorTime = millis();
12   float h = dht.readHumidity();
13   float t = dht.readTemperature();
14
15   if (isnan(h) || isnan(t)) {
16     currentHum = String(h, 1);
17     currentTemp = String(t, 1);
18     notifyClients();
19   }
20 }
```

2. Di Sisi Tampilan HTML

Tambahkan kartu kecil buat nampilin kelembapannya:

```
1 <div class="card">
2   <h2>Kelembapan: <span id="humValue">--</span> %</h2>
3 </div>
4 |
```

3. Di Sisi JavaScript (onMessage)

Tangkap data JSON-nya, lalu suntikkan ke elemen HTML yang punya ID humValue:

```
1 function onMessage(event) {  
2     var data = JSON.parse(event.data);  
3  
4     // Masukin data ke elemen HTML  
5     document.getElementById('temp').innerText = data.suhu;  
6     document.getElementById('humValue').innerText = data.hum;  
7     document.getElementById('ledStatus').innerText = data.led;  
8  
9     // Atur tombol LED  
10    var btn = document.getElementById('btn');  
11    if(data.led === "ON") {  
12        btn.innerText = "Matikan LED";  
13        btn.className = "off";  
14    } else {  
15        btn.innerText = "Nyalakan LED";  
16        btn.className = "";  
17    }  
18 }
```